

APIM Configuration and Policies

Gateway products, JWT policy, roles, routes and deployment.



Summary

This document explains the subject in straightforward professional English, then adds the engineering detail needed to build, operate, troubleshoot or review it. Claims are based on the repository evidence snapshot; missing source details are called out instead of guessed.

Document details

Edition 2.0 | Evidence date: 14 August 2026 | Repository:
rmorampudi09-arch/Craves-Build-platform | Product evidence snapshot: main @
3225f7fa531a6b07185fb5e034f7930f8bd8b571

Summary and how to use this document

Gateway products, JWT policy, roles, routes and deployment. The purpose is to make the system understandable to a new team member while still giving engineers enough detail to trace ownership, data flow, deployment impact and failure handling.

Current implementation

Direct code, README, migration, pipeline or commit evidence exists on the reviewed main snapshot.

Foundation / partial

Useful groundwork exists, but the repository does not prove a complete production runtime for every part.

Legacy / reference

Older implementation is retained for history or compatibility and is not treated as the preferred runtime.

Needs source confirmation

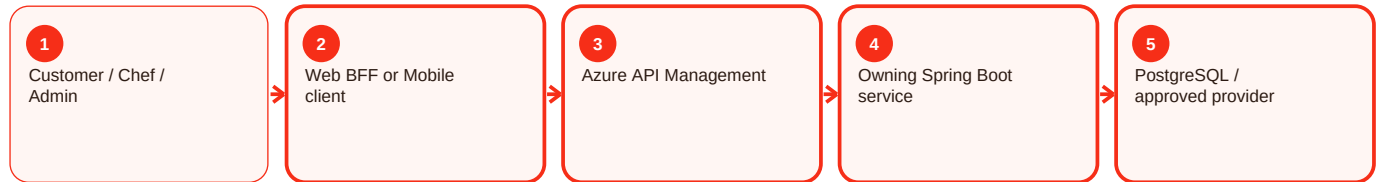
A referenced artifact or exact value could not be verified and is therefore not invented.

Reading order

- Start with the summary to understand the responsibility in normal language.
- Use process diagrams to follow requests across client, APIM, services, data and providers.
- Use the troubleshooting sections to diagnose by evidence rather than by retrying blindly.
- Use deployment and validation sections before or after changing a component.
- Use evidence status to separate proven behavior from gaps and future work.

How the request moves through Craves

The exact components depend on the feature, but the normal pattern is consistent. The user interface starts the request, the gateway and owning service enforce trust, the service writes authoritative state, and provider integrations remain behind controlled boundaries.



Key rule

A screen can hide or show an action for usability, but that is not security. APIM and the owning backend still validate the caller, role, ownership and current business state. Likewise, provider success is not accepted as Craves state until the backend verifies and records the result.

Gateway role - Overview and responsibility

Current implementation

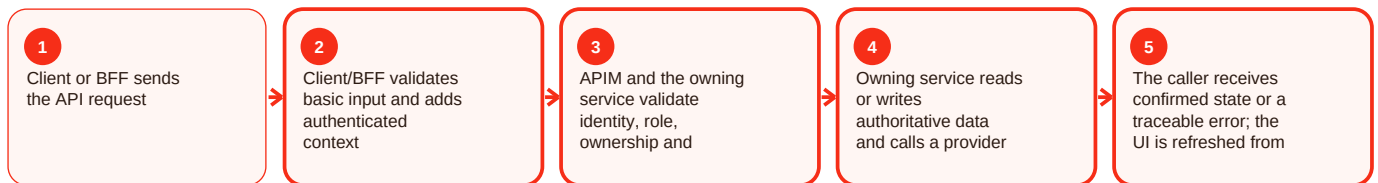
Summary

Gateway role is part of the apim area of Craves. Azure API Management is the intended policy and routing front door between clients/BFFs and backend services. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The APIM README describes craves-customer without APIM subscription requirement and craves-admin with subscription required, JWT validation against Entra and operation-level role checks from named values. Its cited OpenAPI path was not resolvable on the reviewed main snapshot and is documented as a gap. For gateway role, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 at gateway: JWT validation failed. Resolution: Verify token source, issuer/audience and APIM policy.

403 at gateway: Role/product policy denied the caller. Resolution: Verify admin/customer product and operation role rules.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: `infra/apim/README.md`. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Gateway role - Functional behavior and data

Current implementation

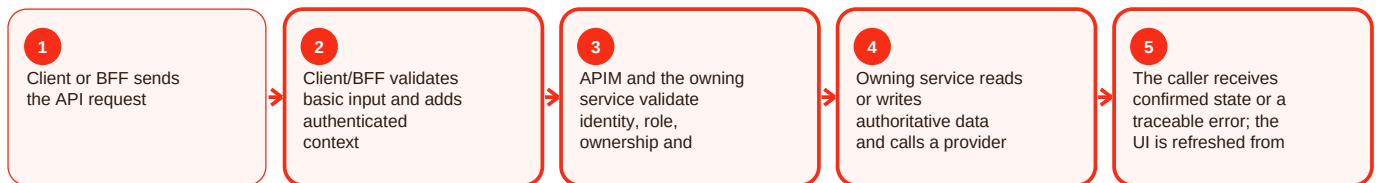
Summary

Gateway role is part of the apim area of Craves. Azure API Management is the intended policy and routing front door between clients/BFFs and backend services. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The APIM README describes craves-customer without APIM subscription requirement and craves-admin with subscription required, JWT validation against Entra and operation-level role checks from named values. Its cited OpenAPI path was not resolvable on the reviewed main snapshot and is documented as a gap. For gateway role, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 at gateway: JWT validation failed. Resolution: Verify token source, issuer/audience and APIM policy.

403 at gateway: Role/product policy denied the caller. Resolution: Verify admin/customer product and operation role rules.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: [infra/apim/README.md](#). Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Gateway role - Operations, errors and deployment

Current implementation

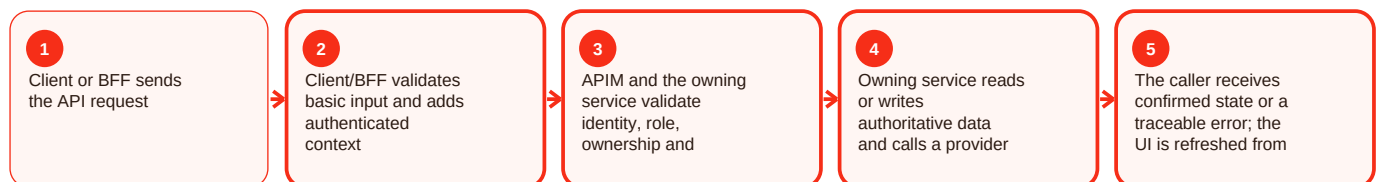
Summary

Gateway role is part of the apim area of Craves. Azure API Management is the intended policy and routing front door between clients/BFFs and backend services. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The APIM README describes craves-customer without APIM subscription requirement and craves-admin with subscription required, JWT validation against Entra and operation-level role checks from named values. Its cited OpenAPI path was not resolvable on the reviewed main snapshot and is documented as a gap. For gateway role, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 at gateway: JWT validation failed. Resolution: Verify token source, issuer/audience and APIM policy.

403 at gateway: Role/product policy denied the caller. Resolution: Verify admin/customer product and operation role rules.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: `infra/apim/README.md`. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Customer product - Overview and responsibility

Current implementation

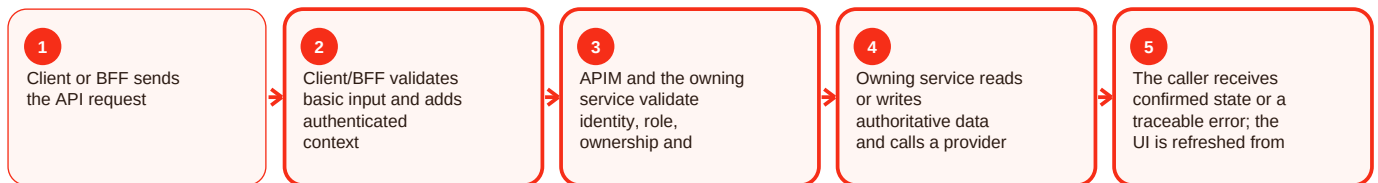
Summary

Customer product is part of the apim area of Craves. Azure API Management is the intended policy and routing front door between clients/BFFs and backend services. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The APIM README describes craves-customer without APIM subscription requirement and craves-admin with subscription required, JWT validation against Entra and operation-level role checks from named values. Its cited OpenAPI path was not resolvable on the reviewed main snapshot and is documented as a gap. For customer product, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 at gateway: JWT validation failed. Resolution: Verify token source, issuer/audience and APIM policy.

403 at gateway: Role/product policy denied the caller. Resolution: Verify admin/customer product and operation role rules.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: `infra/apim/README.md`. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Customer product - Functional behavior and data

Current implementation

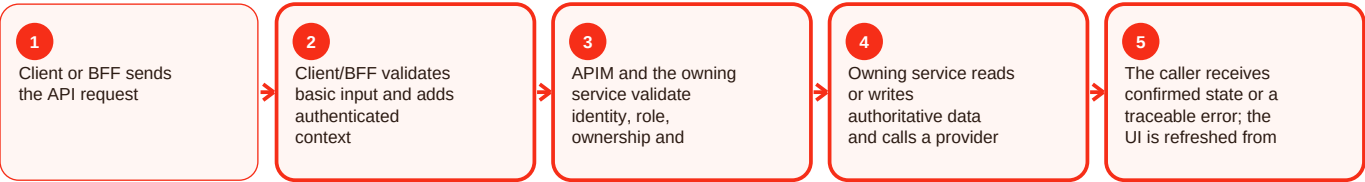
Summary

Customer product is part of the apim area of Craves. Azure API Management is the intended policy and routing front door between clients/BFFs and backend services. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The APIM README describes craves-customer without APIM subscription requirement and craves-admin with subscription required, JWT validation against Entra and operation-level role checks from named values. Its cited OpenAPI path was not resolvable on the reviewed main snapshot and is documented as a gap. For customer product, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 at gateway: JWT validation failed. Resolution: Verify token source, issuer/audience and APIM policy.

403 at gateway: Role/product policy denied the caller. Resolution: Verify admin/customer product and operation role rules.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: `infra/apim/README.md`. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Customer product - Operations, errors and deployment

Current implementation

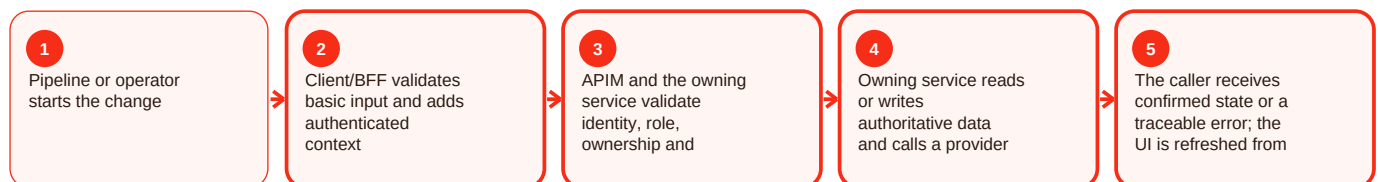
Summary

Customer product is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For customer product, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Admin product - Overview and responsibility

Current implementation

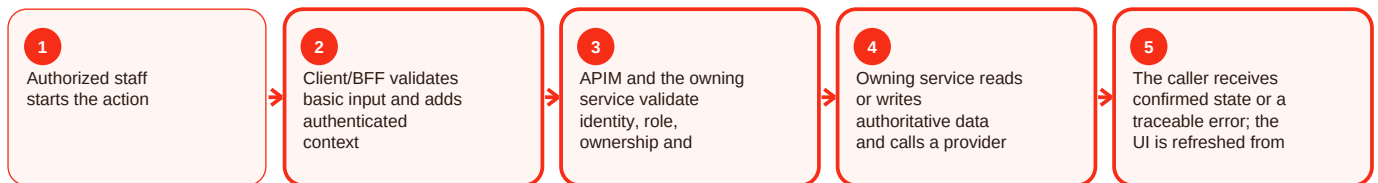
Summary

Admin product is part of the admin area of Craves. Administrative tooling lets authorized staff review marketplace state unavailable to ordinary customers and chefs. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The repository contains legacy apps/admin plus newer customer-web-next admin/support/chef-review modules. APIM guidance places admin operations in a subscription-protected product with additional role checks. For admin product, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Admin action denied: Caller lacks admin role/product permission. Resolution: Use approved staff access process and least privilege.

Operation changes unexpected data: Wrong record/state selected. Resolution: Stop, capture audit/request evidence and follow the documented recovery path.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/admin/README.md; apps/customer-web-next/src/features/admin/; apps/customer-web-next/src/features/chef-review/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Admin product - Functional behavior and data

Current implementation

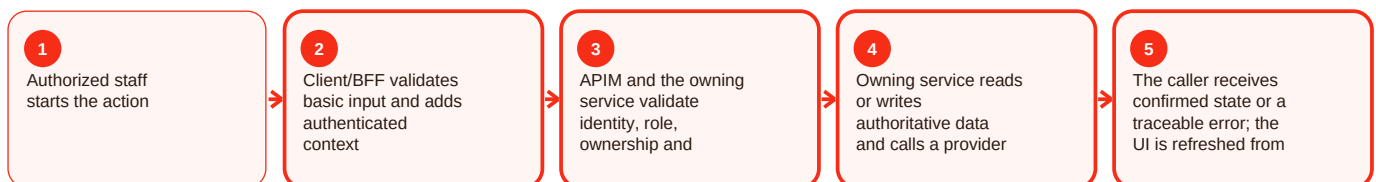
Summary

Admin product is part of the admin area of Craves. Administrative tooling lets authorized staff review marketplace state unavailable to ordinary customers and chefs. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The repository contains legacy apps/admin plus newer customer-web-next admin/support/chef-review modules. APIM guidance places admin operations in a subscription-protected product with additional role checks. For admin product, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Admin action denied: Caller lacks admin role/product permission. Resolution: Use approved staff access process and least privilege.

Operation changes unexpected data: Wrong record/state selected. Resolution: Stop, capture audit/request evidence and follow the documented recovery path.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/admin/README.md; apps/customer-web-next/src/features/admin/; apps/customer-web-next/src/features/chef-review/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Admin product - Operations, errors and deployment

Current implementation

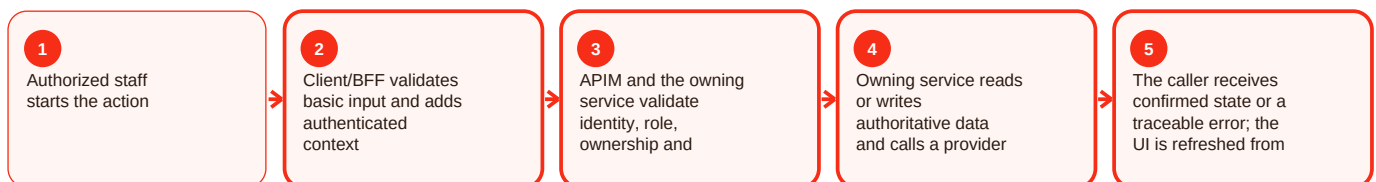
Summary

Admin product is part of the admin area of Craves. Administrative tooling lets authorized staff review marketplace state unavailable to ordinary customers and chefs. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The repository contains legacy apps/admin plus newer customer-web-next admin/support/chef-review modules. APIM guidance places admin operations in a subscription-protected product with additional role checks. For admin product, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Admin action denied: Caller lacks admin role/product permission. Resolution: Use approved staff access process and least privilege.

Operation changes unexpected data: Wrong record/state selected. Resolution: Stop, capture audit/request evidence and follow the documented recovery path.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/admin/README.md; apps/customer-web-next/src/features/admin/; apps/customer-web-next/src/features/chef-review/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Subscription keys - Overview and responsibility

Current implementation

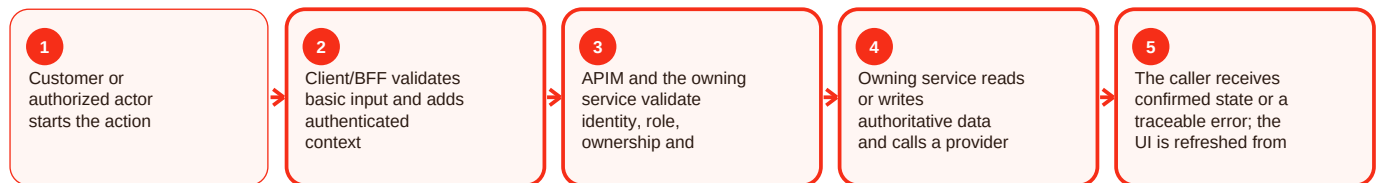
Summary

Subscription keys is part of the subscription area of Craves. Subscriptions support repeat service and entitlements on top of one-off ordering. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

subscription-service migrations cover plans, subscriptions, payment identifiers/tokens, idempotency, request fingerprints and usage counters. The latest observed main commit adds shared entitlements and gated services. For subscription keys, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Lifecycle action rejected: Current subscription state does not allow it. Resolution: Reload current state and use a supported transition.

Commercial rule missing: Product/Finance decision is intentionally not defined. Resolution: Document the decision before implementing it.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/subscription-service/README.md; apps/customer-web-next/src/components/plans/; apps/customer-web-next/src/components/subscriptions/; commit 3225f7fa. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Subscription keys - Functional behavior and data

Current implementation

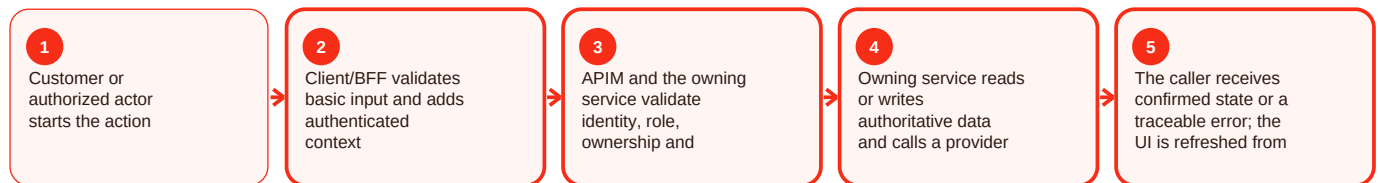
Summary

Subscription keys is part of the subscription area of Craves. Subscriptions support repeat service and entitlements on top of one-off ordering. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

subscription-service migrations cover plans, subscriptions, payment identifiers/tokens, idempotency, request fingerprints and usage counters. The latest observed main commit adds shared entitlements and gated services. For subscription keys, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Lifecycle action rejected: Current subscription state does not allow it. Resolution: Reload current state and use a supported transition.

Commercial rule missing: Product/Finance decision is intentionally not defined. Resolution: Document the decision before implementing it.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/subscription-service/README.md; apps/customer-web-next/src/components/plans/; apps/customer-web-next/src/components/subscriptions/; commit 3225f7fa. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Subscription keys - Operations, errors and deployment

Current implementation

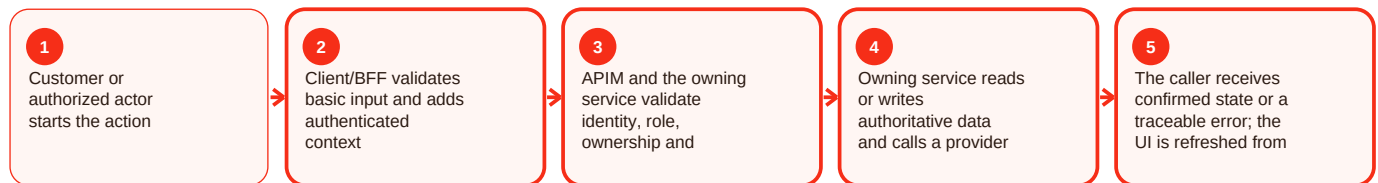
Summary

Subscription keys is part of the subscription area of Craves. Subscriptions support repeat service and entitlements on top of one-off ordering. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

subscription-service migrations cover plans, subscriptions, payment identifiers/tokens, idempotency, request fingerprints and usage counters. The latest observed main commit adds shared entitlements and gated services. For subscription keys, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Lifecycle action rejected: Current subscription state does not allow it. Resolution: Reload current state and use a supported transition.

Commercial rule missing: Product/Finance decision is intentionally not defined. Resolution: Document the decision before implementing it.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/subscription-service/README.md; apps/customer-web-next/src/components/plans/; apps/customer-web-next/src/components/subscriptions/; commit 3225f7fa. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Jwt validation - Overview and responsibility

Current implementation

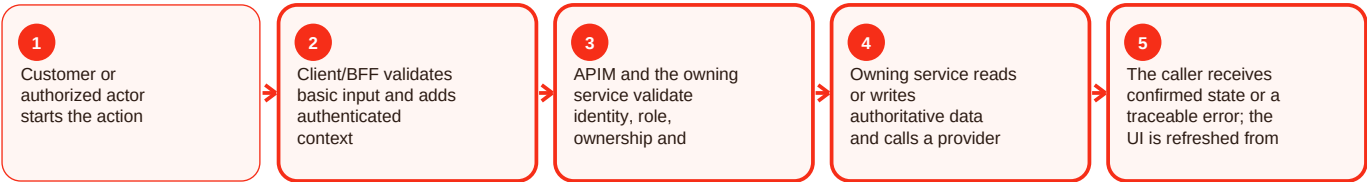
Summary

Jwt validation is part of the auth area of Craves. Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory. For JWT validation, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Sign-in fails: OTP/session/token is invalid, expired or misconfigured. Resolution: Use the supported sign-in or refresh path and check issuer/audience/configuration.

403 after sign-in: Caller lacks the required role or ownership. Resolution: Confirm intended access; do not bypass authorization.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Jwt validation - Functional behavior and data

Current implementation

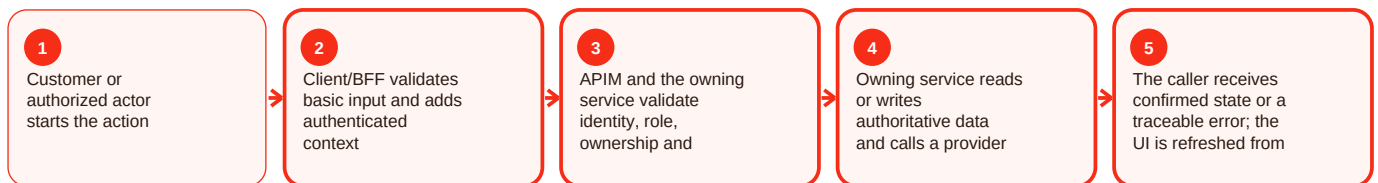
Summary

Jwt validation is part of the auth area of Craves. Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory. For JWT validation, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Sign-in fails: OTP/session/token is invalid, expired or misconfigured. Resolution: Use the supported sign-in or refresh path and check issuer/audience/configuration.

403 after sign-in: Caller lacks the required role or ownership. Resolution: Confirm intended access; do not bypass authorization.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Jwt validation - Operations, errors and deployment

Current implementation

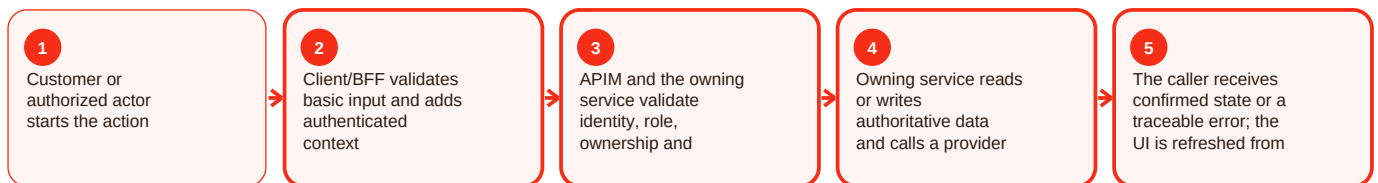
Summary

Jwt validation is part of the auth area of Craves. Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory. For JWT validation, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Sign-in fails: OTP/session/token is invalid, expired or misconfigured. Resolution: Use the supported sign-in or refresh path and check issuer/audience/configuration.

403 after sign-in: Caller lacks the required role or ownership. Resolution: Confirm intended access; do not bypass authorization.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Entra tenant - Overview and responsibility

Current implementation

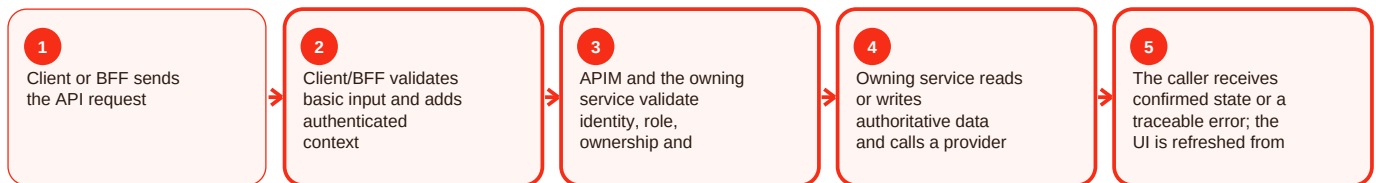
Summary

Entra tenant is part of the apim area of Craves. Azure API Management is the intended policy and routing front door between clients/BFFs and backend services. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The APIM README describes craves-customer without APIM subscription requirement and craves-admin with subscription required, JWT validation against Entra and operation-level role checks from named values. Its cited OpenAPI path was not resolvable on the reviewed main snapshot and is documented as a gap. For Entra tenant, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 at gateway: JWT validation failed. Resolution: Verify token source, issuer/audience and APIM policy.

403 at gateway: Role/product policy denied the caller. Resolution: Verify admin/customer product and operation role rules.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: `infra/apim/README.md`. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Entra tenant - Functional behavior and data

Current implementation

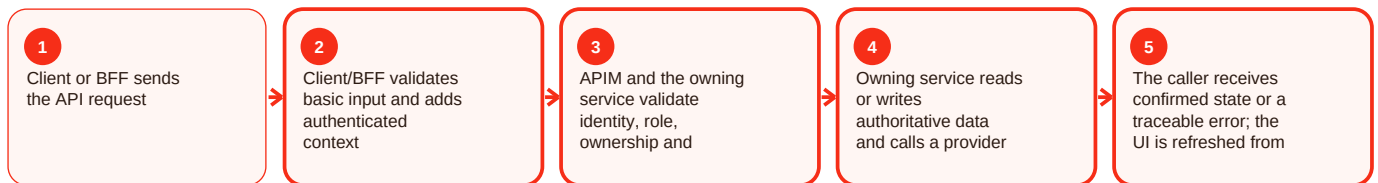
Summary

Entra tenant is part of the apim area of Craves. Azure API Management is the intended policy and routing front door between clients/BFFs and backend services. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The APIM README describes craves-customer without APIM subscription requirement and craves-admin with subscription required, JWT validation against Entra and operation-level role checks from named values. Its cited OpenAPI path was not resolvable on the reviewed main snapshot and is documented as a gap. For Entra tenant, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 at gateway: JWT validation failed. Resolution: Verify token source, issuer/audience and APIM policy.

403 at gateway: Role/product policy denied the caller. Resolution: Verify admin/customer product and operation role rules.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: `infra/apim/README.md`. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Entra tenant - Operations, errors and deployment

Current implementation

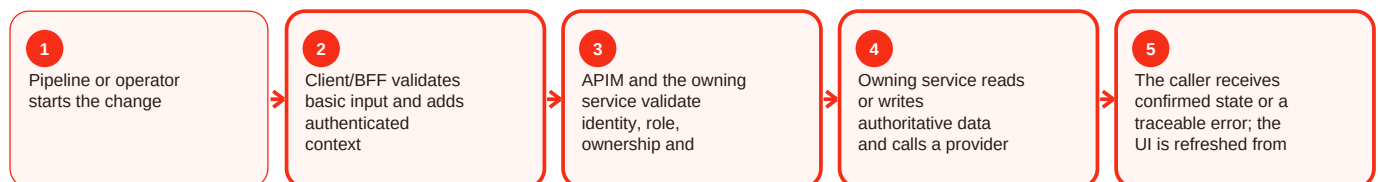
Summary

Entra tenant is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For Entra tenant, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Operation role policy - Overview and responsibility

Current implementation

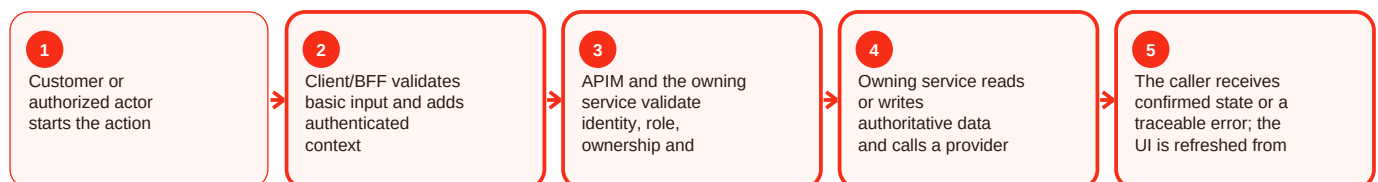
Summary

Operation role policy is part of the security area of Craves. Security is layered: identity proof, token validation, role/ownership checks, secret isolation, provider-signature checks and deployment gates address different failure modes. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

APIM adds gateway policy, services re-check authorization, web uses secure cookies, mobile milestones use secure token storage, Razorpay callbacks use HMAC verification, and local starter secrets are prohibited in deployed Azure profiles where documented. For operation role policy, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Secret exposed: Sensitive value appeared outside secret storage. Resolution: Rotate, remove exposure and record incident evidence.

Spoofed callback/internal call: Signature/internal credential validation failed. Resolution: Fail closed and restore verification before re-enabling.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: `infra/apim/README.md`; `services/user-chef-service/README.md`; `services/order-service/README.md`; `apps/customer-web-next/README.md`. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Operation role policy - Functional behavior and data

Current implementation

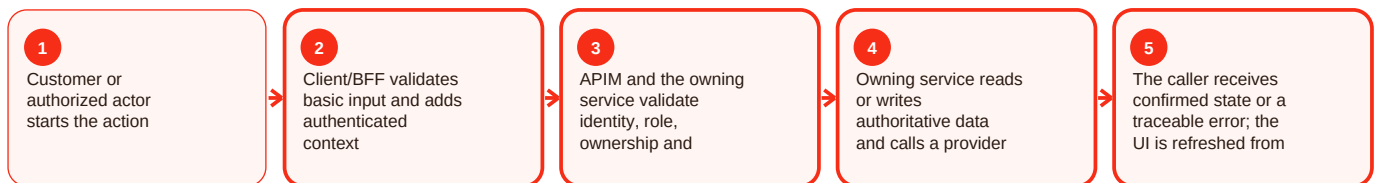
Summary

Operation role policy is part of the security area of Craves. Security is layered: identity proof, token validation, role/ownership checks, secret isolation, provider-signature checks and deployment gates address different failure modes. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

APIM adds gateway policy, services re-check authorization, web uses secure cookies, mobile milestones use secure token storage, Razorpay callbacks use HMAC verification, and local starter secrets are prohibited in deployed Azure profiles where documented. For operation role policy, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Secret exposed: Sensitive value appeared outside secret storage. Resolution: Rotate, remove exposure and record incident evidence.

Spoofed callback/internal call: Signature/internal credential validation failed. Resolution: Fail closed and restore verification before re-enabling.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: `infra/apim/README.md`; `services/user-chef-service/README.md`; `services/order-service/README.md`; `apps/customer-web-next/README.md`. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Operation role policy - Operations, errors and deployment

Current implementation

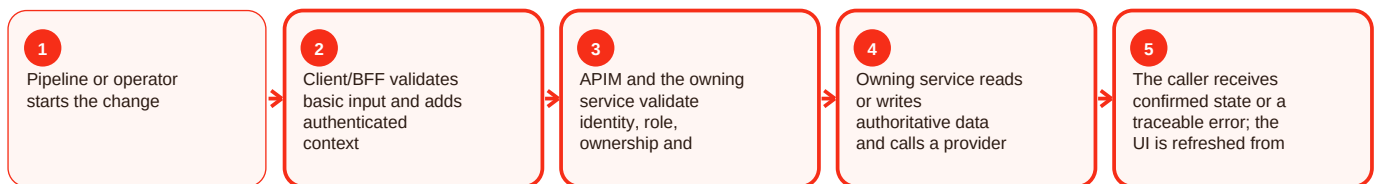
Summary

Operation role policy is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For operation role policy, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed.

Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Named values - Overview and responsibility

Current implementation

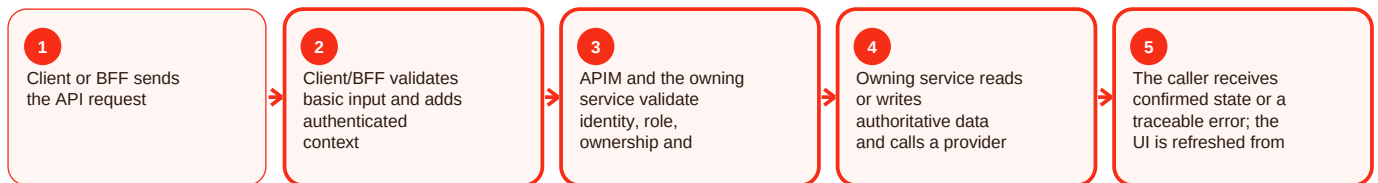
Summary

Named values is part of the apim area of Craves. Azure API Management is the intended policy and routing front door between clients/BFFs and backend services. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The APIM README describes craves-customer without APIM subscription requirement and craves-admin with subscription required, JWT validation against Entra and operation-level role checks from named values. Its cited OpenAPI path was not resolvable on the reviewed main snapshot and is documented as a gap. For named values, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 at gateway: JWT validation failed. Resolution: Verify token source, issuer/audience and APIM policy.

403 at gateway: Role/product policy denied the caller. Resolution: Verify admin/customer product and operation role rules.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: `infra/apim/README.md`. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Named values - Functional behavior and data

Current implementation

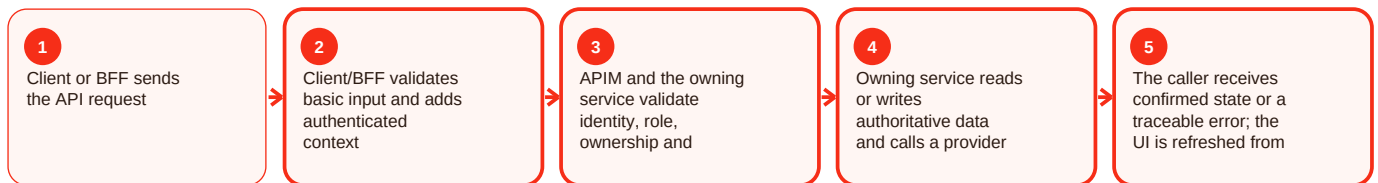
Summary

Named values is part of the apim area of Craves. Azure API Management is the intended policy and routing front door between clients/BFFs and backend services. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The APIM README describes craves-customer without APIM subscription requirement and craves-admin with subscription required, JWT validation against Entra and operation-level role checks from named values. Its cited OpenAPI path was not resolvable on the reviewed main snapshot and is documented as a gap. For named values, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 at gateway: JWT validation failed. Resolution: Verify token source, issuer/audience and APIM policy.

403 at gateway: Role/product policy denied the caller. Resolution: Verify admin/customer product and operation role rules.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: `infra/apim/README.md`. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Named values - Operations, errors and deployment

Current implementation

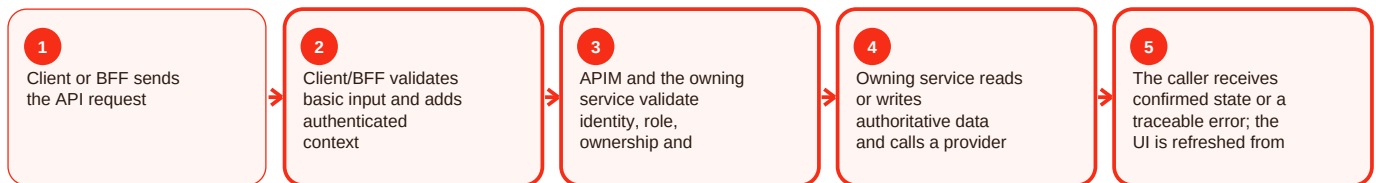
Summary

Named values is part of the apim area of Craves. Azure API Management is the intended policy and routing front door between clients/BFFs and backend services. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The APIM README describes craves-customer without APIM subscription requirement and craves-admin with subscription required, JWT validation against Entra and operation-level role checks from named values. Its cited OpenAPI path was not resolvable on the reviewed main snapshot and is documented as a gap. For named values, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 at gateway: JWT validation failed. Resolution: Verify token source, issuer/audience and APIM policy.

403 at gateway: Role/product policy denied the caller. Resolution: Verify admin/customer product and operation role rules.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: `infra/apim/README.md`. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Contract-admins - Overview and responsibility

Current implementation

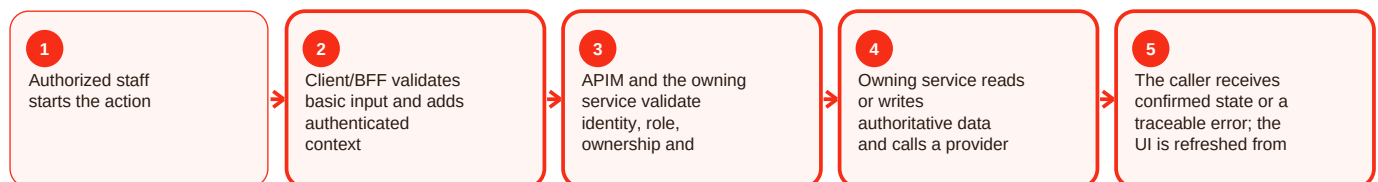
Summary

Contract-admins is part of the admin area of Craves. Administrative tooling lets authorized staff review marketplace state unavailable to ordinary customers and chefs. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The repository contains legacy apps/admin plus newer customer-web-next admin/support/chef-review modules. APIM guidance places admin operations in a subscription-protected product with additional role checks. For contract-admins, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Admin action denied: Caller lacks admin role/product permission. Resolution: Use approved staff access process and least privilege.

Operation changes unexpected data: Wrong record/state selected. Resolution: Stop, capture audit/request evidence and follow the documented recovery path.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/admin/README.md; apps/customer-web-next/src/features/admin/; apps/customer-web-next/src/features/chef-review/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Contract-admins - Functional behavior and data

Current implementation

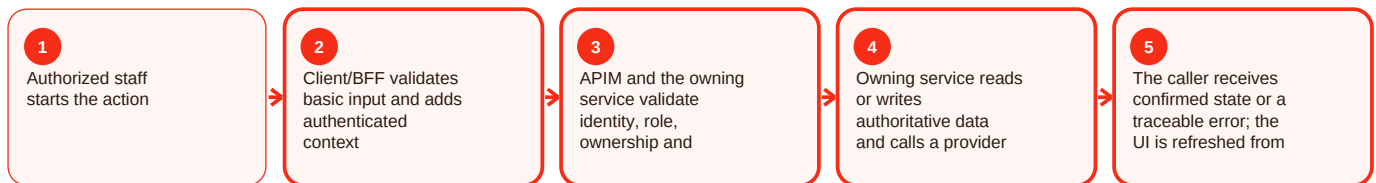
Summary

Contract-admins is part of the admin area of Craves. Administrative tooling lets authorized staff review marketplace state unavailable to ordinary customers and chefs. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The repository contains legacy apps/admin plus newer customer-web-next admin/support/chef-review modules. APIM guidance places admin operations in a subscription-protected product with additional role checks. For contract-admins, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Admin action denied: Caller lacks admin role/product permission. Resolution: Use approved staff access process and least privilege.

Operation changes unexpected data: Wrong record/state selected. Resolution: Stop, capture audit/request evidence and follow the documented recovery path.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/admin/README.md; apps/customer-web-next/src/features/admin/; apps/customer-web-next/src/features/chef-review/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Contract-admins - Operations, errors and deployment

Current implementation

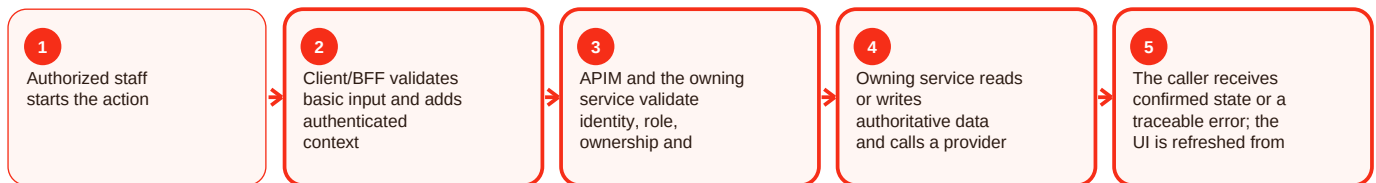
Summary

Contract-admins is part of the admin area of Craves. Administrative tooling lets authorized staff review marketplace state unavailable to ordinary customers and chefs. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The repository contains legacy apps/admin plus newer customer-web-next admin/support/chef-review modules. APIM guidance places admin operations in a subscription-protected product with additional role checks. For contract-admins, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Admin action denied: Caller lacks admin role/product permission. Resolution: Use approved staff access process and least privilege.

Operation changes unexpected data: Wrong record/state selected. Resolution: Stop, capture audit/request evidence and follow the documented recovery path.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/admin/README.md; apps/customer-web-next/src/features/admin/; apps/customer-web-next/src/features/chef-review/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Status / settings / restart - Overview and responsibility

Current implementation

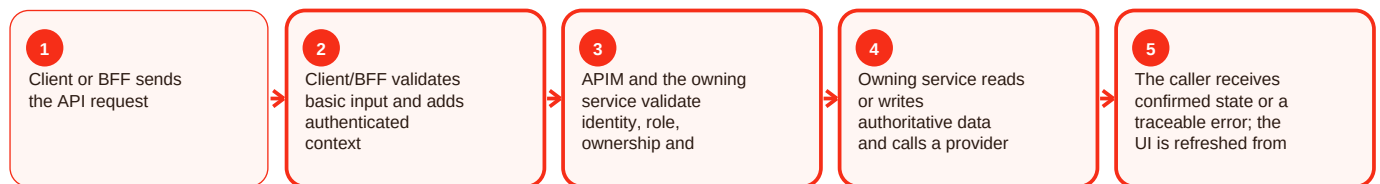
Summary

Status / settings / restart is part of the apim area of Craves. Azure API Management is the intended policy and routing front door between clients/BFFs and backend services. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The APIM README describes craves-customer without APIM subscription requirement and craves-admin with subscription required, JWT validation against Entra and operation-level role checks from named values. Its cited OpenAPI path was not resolvable on the reviewed main snapshot and is documented as a gap. For status/settings/restart, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 at gateway: JWT validation failed. Resolution: Verify token source, issuer/audience and APIM policy.

403 at gateway: Role/product policy denied the caller. Resolution: Verify admin/customer product and operation role rules.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Status / settings / restart - Functional behavior and data

Current implementation

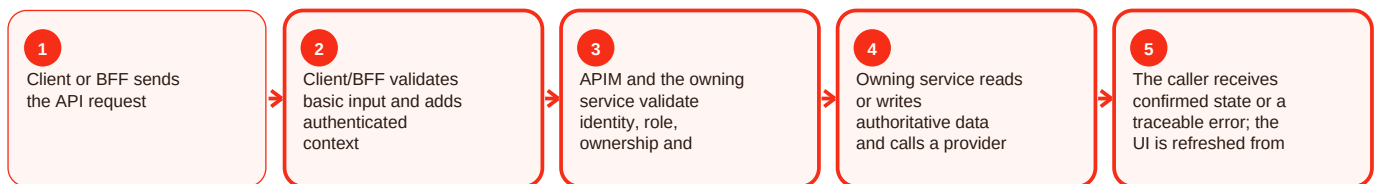
Summary

Status / settings / restart is part of the apim area of Craves. Azure API Management is the intended policy and routing front door between clients/BFFs and backend services. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The APIM README describes craves-customer without APIM subscription requirement and craves-admin with subscription required, JWT validation against Entra and operation-level role checks from named values. Its cited OpenAPI path was not resolvable on the reviewed main snapshot and is documented as a gap. For status/settings/restart, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 at gateway: JWT validation failed. Resolution: Verify token source, issuer/audience and APIM policy.

403 at gateway: Role/product policy denied the caller. Resolution: Verify admin/customer product and operation role rules.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Status / settings / restart - Operations, errors and deployment

Current implementation

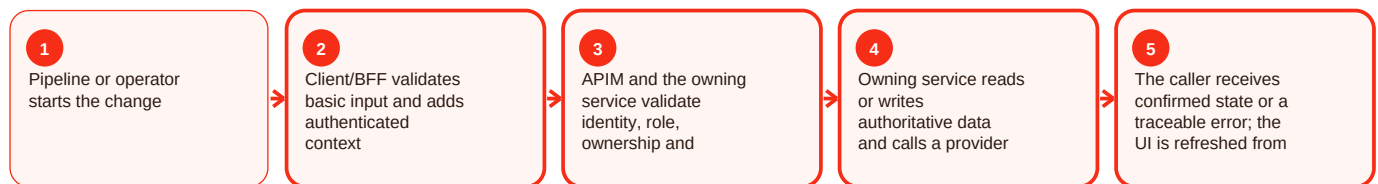
Summary

Status / settings / restart is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For status/settings/restart, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed.

Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Catalog routes - Overview and responsibility

Current implementation

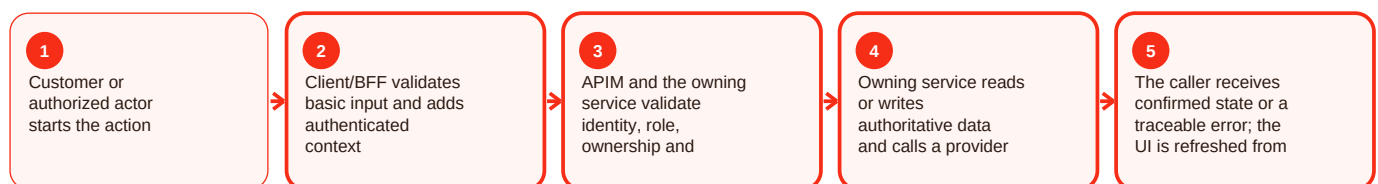
Summary

Catalog routes is part of the catalog area of Craves. Catalog is the source of browseable meal and cuisine information used by discovery and commerce validation. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

catalog-service is a Spring Boot/Flyway service with JWT controls. The web contains discovery, meals, search and selector modules. Administrative catalog operations are described by APIM guidance, including bulk pause/import and SHA-256 integrity checks. For catalog routes, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Nearby results empty: No active geocoded kitchen with sellable items in query radius. Resolution: Confirm kitchen coordinates, active menu items and caller radius.

Menu item cannot sell: Packaging/availability/status metadata incomplete. Resolution: Chef must provide valid required metadata and make the item available.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/catalog-service/README.md; apps/customer-web-next/src/components/discovery/; apps/customer-web-next/src/components/meals/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Catalog routes - Functional behavior and data

Current implementation

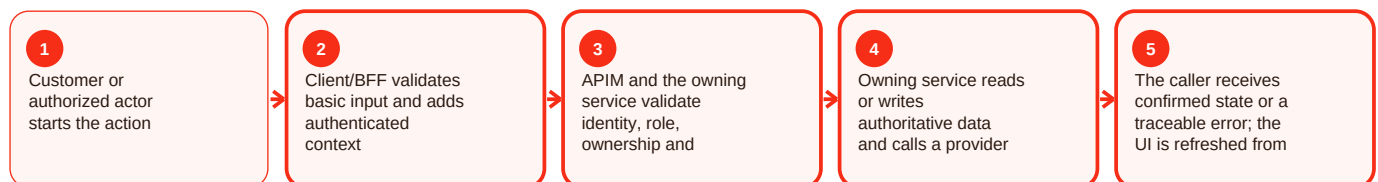
Summary

Catalog routes is part of the catalog area of Craves. Catalog is the source of browseable meal and cuisine information used by discovery and commerce validation. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

catalog-service is a Spring Boot/Flyway service with JWT controls. The web contains discovery, meals, search and selector modules. Administrative catalog operations are described by APIM guidance, including bulk pause/import and SHA-256 integrity checks. For catalog routes, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Nearby results empty: No active geocoded kitchen with sellable items in query radius. Resolution: Confirm kitchen coordinates, active menu items and caller radius.

Menu item cannot sell: Packaging/availability/status metadata incomplete. Resolution: Chef must provide valid required metadata and make the item available.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/catalog-service/README.md; apps/customer-web-next/src/components/discovery/; apps/customer-web-next/src/components/meals/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Catalog routes - Operations, errors and deployment

Current implementation

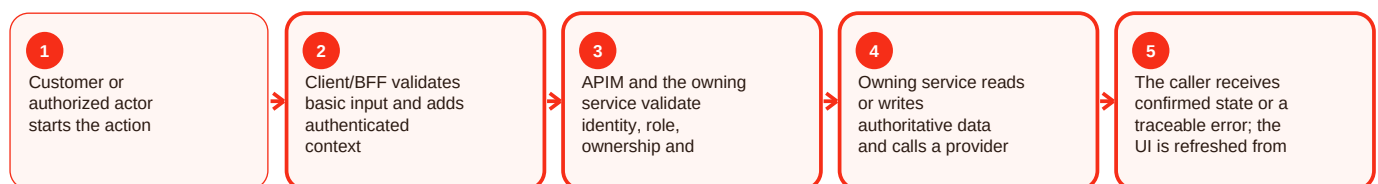
Summary

Catalog routes is part of the catalog area of Craves. Catalog is the source of browseable meal and cuisine information used by discovery and commerce validation. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

catalog-service is a Spring Boot/Flyway service with JWT controls. The web contains discovery, meals, search and selector modules. Administrative catalog operations are described by APIM guidance, including bulk pause/import and SHA-256 integrity checks. For catalog routes, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Nearby results empty: No active geocoded kitchen with sellable items in query radius. Resolution: Confirm kitchen coordinates, active menu items and caller radius.

Menu item cannot sell: Packaging/availability/status metadata incomplete. Resolution: Chef must provide valid required metadata and make the item available.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/catalog-service/README.md; apps/customer-web-next/src/components/discovery/; apps/customer-web-next/src/components/meals/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Bulk pause / import - Overview and responsibility

Current implementation

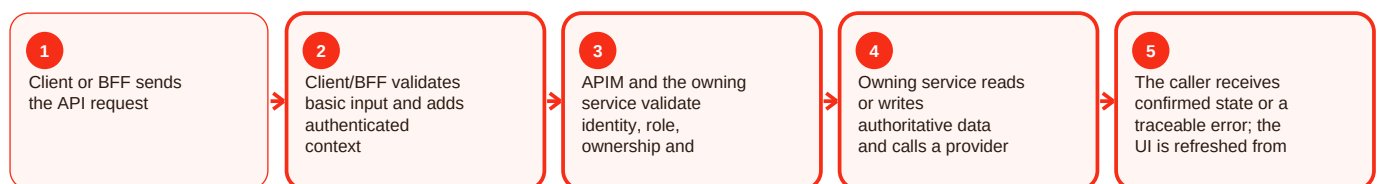
Summary

Bulk pause / import is part of the apim area of Craves. Azure API Management is the intended policy and routing front door between clients/BFFs and backend services. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The APIM README describes craves-customer without APIM subscription requirement and craves-admin with subscription required, JWT validation against Entra and operation-level role checks from named values. Its cited OpenAPI path was not resolvable on the reviewed main snapshot and is documented as a gap. For bulk pause/import, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 at gateway: JWT validation failed. Resolution: Verify token source, issuer/audience and APIM policy.

403 at gateway: Role/product policy denied the caller. Resolution: Verify admin/customer product and operation role rules.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Bulk pause / import - Functional behavior and data

Current implementation

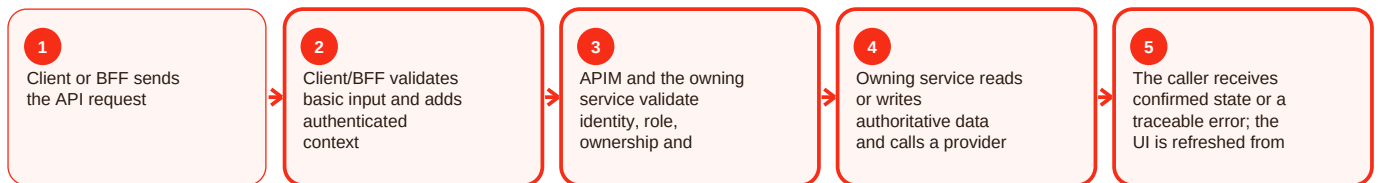
Summary

Bulk pause / import is part of the apim area of Craves. Azure API Management is the intended policy and routing front door between clients/BFFs and backend services. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The APIM README describes craves-customer without APIM subscription requirement and craves-admin with subscription required, JWT validation against Entra and operation-level role checks from named values. Its cited OpenAPI path was not resolvable on the reviewed main snapshot and is documented as a gap. For bulk pause/import, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 at gateway: JWT validation failed. Resolution: Verify token source, issuer/audience and APIM policy.

403 at gateway: Role/product policy denied the caller. Resolution: Verify admin/customer product and operation role rules.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Bulk pause / import - Operations, errors and deployment

Current implementation

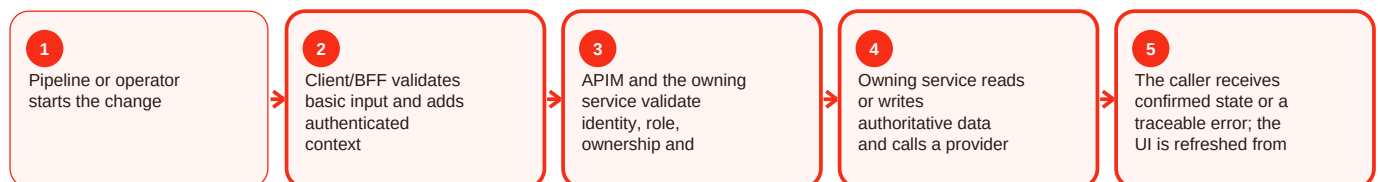
Summary

Bulk pause / import is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For bulk pause/import, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Sha-256 integrity - Overview and responsibility

Current implementation

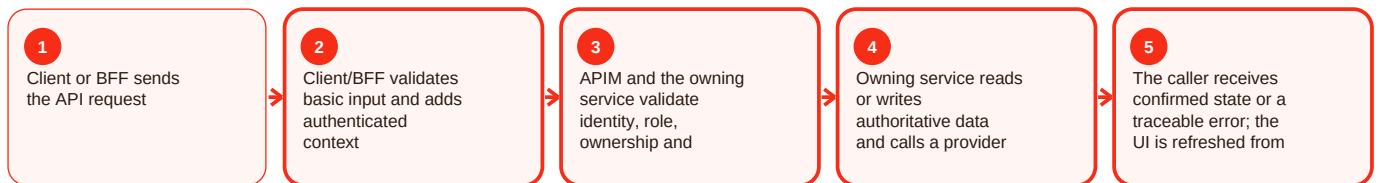
Summary

Sha-256 integrity is part of the apim area of Craves. Azure API Management is the intended policy and routing front door between clients/BFFs and backend services. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The APIM README describes craves-customer without APIM subscription requirement and craves-admin with subscription required, JWT validation against Entra and operation-level role checks from named values. Its cited OpenAPI path was not resolvable on the reviewed main snapshot and is documented as a gap. For SHA-256 integrity, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 at gateway: JWT validation failed. Resolution: Verify token source, issuer/audience and APIM policy.

403 at gateway: Role/product policy denied the caller. Resolution: Verify admin/customer product and operation role rules.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: `infra/apim/README.md`. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Sha-256 integrity - Functional behavior and data

Current implementation

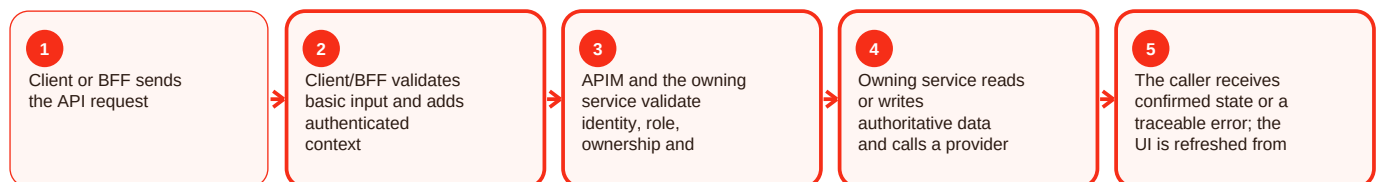
Summary

Sha-256 integrity is part of the apim area of Craves. Azure API Management is the intended policy and routing front door between clients/BFFs and backend services. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The APIM README describes craves-customer without APIM subscription requirement and craves-admin with subscription required, JWT validation against Entra and operation-level role checks from named values. Its cited OpenAPI path was not resolvable on the reviewed main snapshot and is documented as a gap. For SHA-256 integrity, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 at gateway: JWT validation failed. Resolution: Verify token source, issuer/audience and APIM policy.

403 at gateway: Role/product policy denied the caller. Resolution: Verify admin/customer product and operation role rules.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: `infra/apim/README.md`. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Sha-256 integrity - Operations, errors and deployment

Current implementation

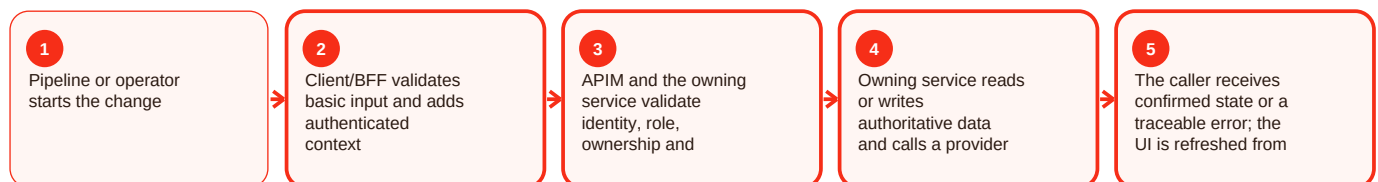
Summary

Sha-256 integrity is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For SHA-256 integrity, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Bff routing - Overview and responsibility

Current implementation

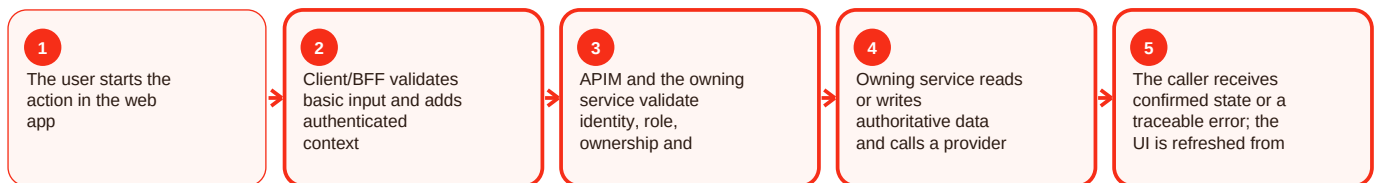
Summary

Bff routing is part of the frontend area of Craves. The primary customer-facing web implementation is apps/customer-web-next, a Next.js 14 App Router application containing customer, chef and newer admin/support modules. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The browser uses same-origin BFF handlers, secure HTTP-only token cookies and APIM at /api/v1. Firebase phone OTP is used for the current web sign-in flow, and backend responses remain authoritative for price, availability and order state. For BFF routing, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Generic upstream error: BFF rejected malformed/unexpected backend response. Resolution: Use request ID and compare the current response contract.

Amount differs: Client state is stale. Resolution: Render backend-authoritative cart/quote state.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/app/; apps/customer-web-next/src/components/; apps/customer-web-next/src/features/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Bff routing - Functional behavior and data

Current implementation

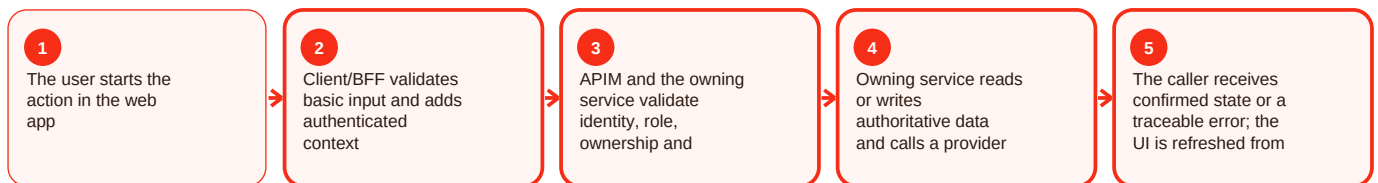
Summary

Bff routing is part of the frontend area of Craves. The primary customer-facing web implementation is apps/customer-web-next, a Next.js 14 App Router application containing customer, chef and newer admin/support modules. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The browser uses same-origin BFF handlers, secure HTTP-only token cookies and APIM at /api/v1. Firebase phone OTP is used for the current web sign-in flow, and backend responses remain authoritative for price, availability and order state. For BFF routing, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Generic upstream error: BFF rejected malformed/unexpected backend response. Resolution: Use request ID and compare the current response contract.

Amount differs: Client state is stale. Resolution: Render backend-authoritative cart/quote state.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/app/; apps/customer-web-next/src/components/; apps/customer-web-next/src/features/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Bff routing - Operations, errors and deployment

Current implementation

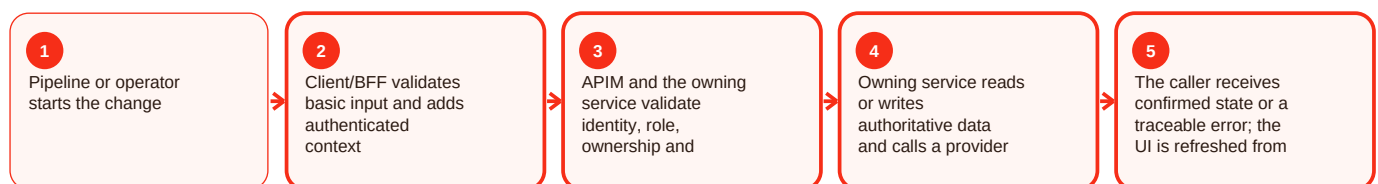
Summary

Bff routing is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For BFF routing, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Mobile routing - Overview and responsibility

Foundation / partial

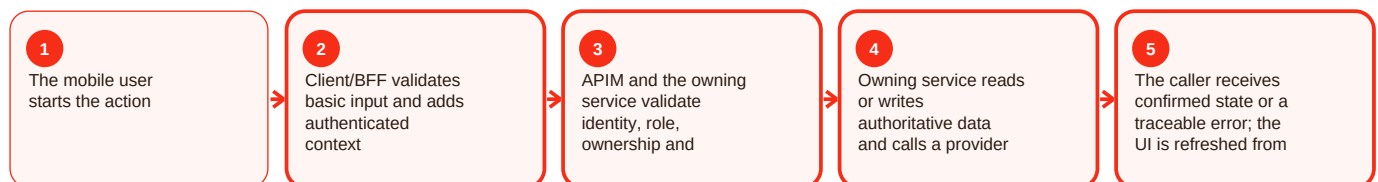
Summary

Mobile routing is part of the mobile area of Craves. Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced. For mobile routing, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 after expiry: Access token expired. Resolution: Attempt the documented silent refresh once, then interactive sign-in if needed.

Build/release cannot be proven: Native source/signing is incomplete in reviewed main. Resolution: Restore the authoritative native project before claiming release readiness.

Validation and evidence

The repository proves contracts and milestone groundwork, but not a complete native runtime on the reviewed main snapshot. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Mobile routing - Functional behavior and data

Foundation / partial

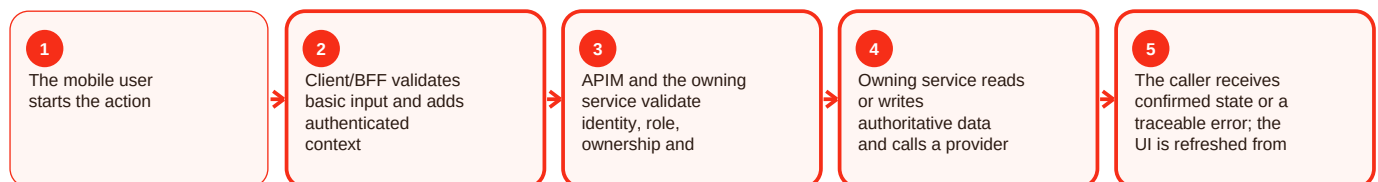
Summary

Mobile routing is part of the mobile area of Craves. Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced. For mobile routing, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 after expiry: Access token expired. Resolution: Attempt the documented silent refresh once, then interactive sign-in if needed.

Build/release cannot be proven: Native source/signing is incomplete in reviewed main. Resolution: Restore the authoritative native project before claiming release readiness.

Validation and evidence

The repository proves contracts and milestone groundwork, but not a complete native runtime on the reviewed main snapshot. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Mobile routing - Operations, errors and deployment

Foundation / partial

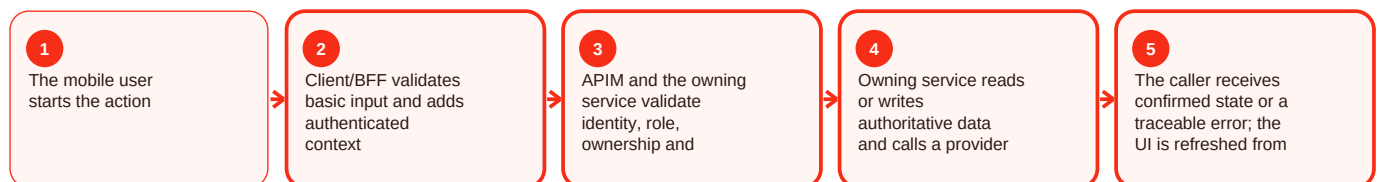
Summary

Mobile routing is part of the mobile area of Craves. Native mobile is proven mainly through milestone identity/API/CI contracts rather than a clearly complete runnable app on the reviewed main snapshot. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The 30 July handover specifies Entra External ID, authorization code with PKCE/MFA, refresh token in Keychain/Keystore, access token in memory, APIM bearer calls, one silent refresh after 401, then interactive sign-in. A short-lived React Native bootstrap branch was referenced. For mobile routing, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 after expiry: Access token expired. Resolution: Attempt the documented silent refresh once, then interactive sign-in if needed.

Build/release cannot be proven: Native source/signing is incomplete in reviewed main. Resolution: Restore the authoritative native project before claiming release readiness.

Validation and evidence

The repository proves contracts and milestone groundwork, but not a complete native runtime on the reviewed main snapshot. Evidence: docs/handover/2026-07-30-customer-mobile-auth-foundation.md; docs/handover/2026-07-30-mobile-ci-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Correlation - Overview and responsibility

Current implementation

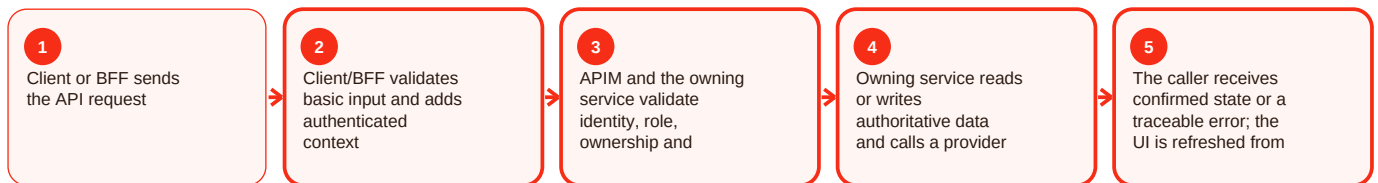
Summary

Correlation is part of the apim area of Craves. Azure API Management is the intended policy and routing front door between clients/BFFs and backend services. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The APIM README describes craves-customer without APIM subscription requirement and craves-admin with subscription required, JWT validation against Entra and operation-level role checks from named values. Its cited OpenAPI path was not resolvable on the reviewed main snapshot and is documented as a gap. For correlation, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 at gateway: JWT validation failed. Resolution: Verify token source, issuer/audience and APIM policy.

403 at gateway: Role/product policy denied the caller. Resolution: Verify admin/customer product and operation role rules.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Correlation - Functional behavior and data

Current implementation

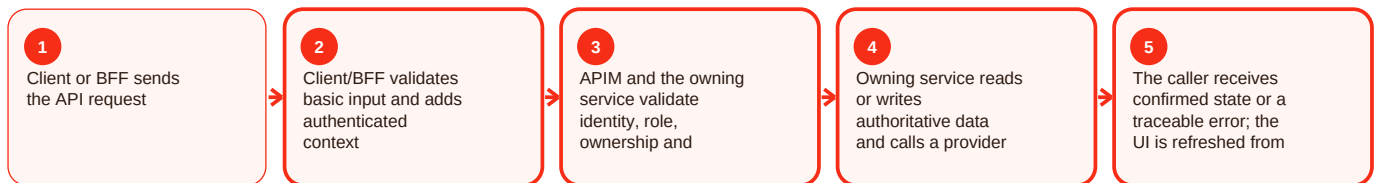
Summary

Correlation is part of the apim area of Craves. Azure API Management is the intended policy and routing front door between clients/BFFs and backend services. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The APIM README describes craves-customer without APIM subscription requirement and craves-admin with subscription required, JWT validation against Entra and operation-level role checks from named values. Its cited OpenAPI path was not resolvable on the reviewed main snapshot and is documented as a gap. For correlation, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 at gateway: JWT validation failed. Resolution: Verify token source, issuer/audience and APIM policy.

403 at gateway: Role/product policy denied the caller. Resolution: Verify admin/customer product and operation role rules.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Correlation - Operations, errors and deployment

Current implementation

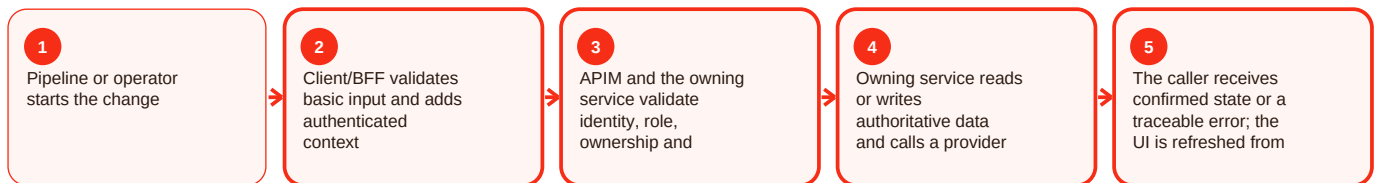
Summary

Correlation is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For correlation, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Problem responses - Overview and responsibility

Current implementation

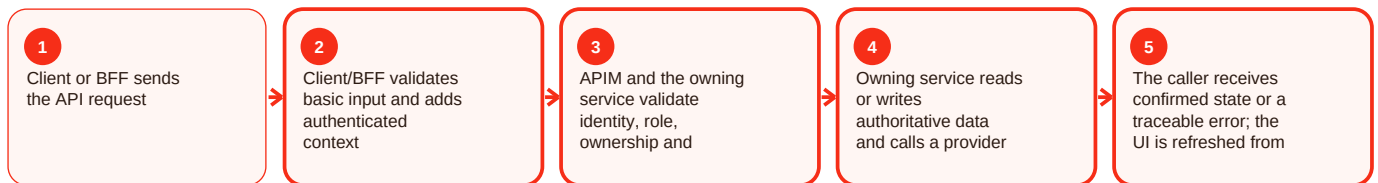
Summary

Problem responses is part of the apim area of Craves. Azure API Management is the intended policy and routing front door between clients/BFFs and backend services. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The APIM README describes craves-customer without APIM subscription requirement and craves-admin with subscription required, JWT validation against Entra and operation-level role checks from named values. Its cited OpenAPI path was not resolvable on the reviewed main snapshot and is documented as a gap. For problem responses, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 at gateway: JWT validation failed. Resolution: Verify token source, issuer/audience and APIM policy.

403 at gateway: Role/product policy denied the caller. Resolution: Verify admin/customer product and operation role rules.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: `infra/apim/README.md`. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Problem responses - Functional behavior and data

Current implementation

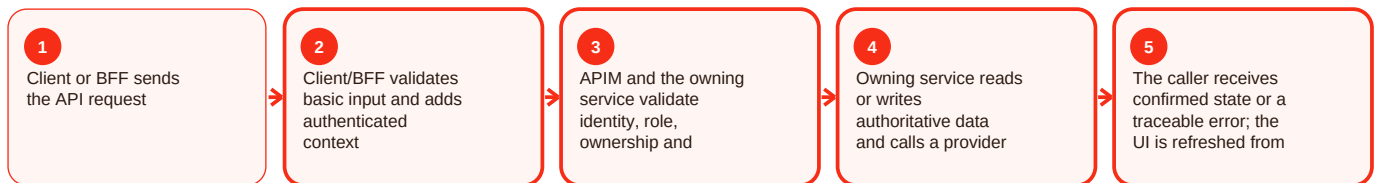
Summary

Problem responses is part of the apim area of Craves. Azure API Management is the intended policy and routing front door between clients/BFFs and backend services. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The APIM README describes craves-customer without APIM subscription requirement and craves-admin with subscription required, JWT validation against Entra and operation-level role checks from named values. Its cited OpenAPI path was not resolvable on the reviewed main snapshot and is documented as a gap. For problem responses, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 at gateway: JWT validation failed. Resolution: Verify token source, issuer/audience and APIM policy.

403 at gateway: Role/product policy denied the caller. Resolution: Verify admin/customer product and operation role rules.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: `infra/apim/README.md`. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Problem responses - Operations, errors and deployment

Current implementation

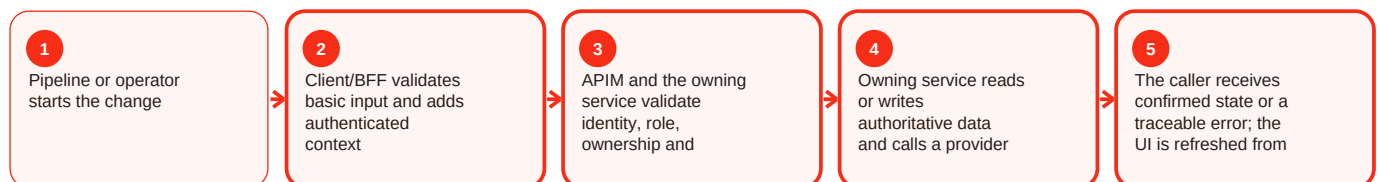
Summary

Problem responses is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For problem responses, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Timeouts / retries - Overview and responsibility

Current implementation

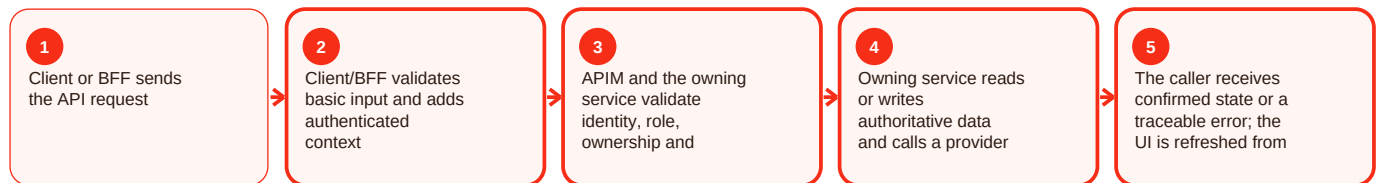
Summary

Timeouts / retries is part of the apim area of Craves. Azure API Management is the intended policy and routing front door between clients/BFFs and backend services. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The APIM README describes craves-customer without APIM subscription requirement and craves-admin with subscription required, JWT validation against Entra and operation-level role checks from named values. Its cited OpenAPI path was not resolvable on the reviewed main snapshot and is documented as a gap. For timeouts/retries, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

401 at gateway: JWT validation failed. Resolution: Verify token source, issuer/audience and APIM policy.

403 at gateway: Role/product policy denied the caller. Resolution: Verify admin/customer product and operation role rules.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: `infra/apim/README.md`. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Operational checklist and documented gaps

Before making a change

- Confirm the intended actor and environment.
- Confirm the current authoritative state before changing anything.
- Capture HTTP status, stable error code and request/correlation ID without secrets.
- Validate the owning component health and required dependency/configuration.
- After the action, reload the state from the backend and confirm the expected result.

Repository gaps that must stay visible

- APIM README cites `infra/apim/craves-openapi.yaml`, but that literal artifact was not resolvable on main during this evidence snapshot.
- `customer-web-next` README names `azure-pipelines-customer-web-next.yml`, but that literal root path was not resolvable; deployment behavior described by the README is retained while trigger/variable details are not invented.
- Native mobile has strong milestone identity/API/CI evidence, but a clearly complete runnable React Native application was not established on the reviewed main snapshot.
- Legacy preview URLs prove historical deployment references rather than current production routing.
- Commercial values such as commissions, payout percentages, subscription pricing, catering thresholds and SLAs are not fabricated where source is silent.

Definition of done

A change is complete only when the intended behavior works, authorization still fails closed, authoritative data is correct, health/readiness are good, monitoring or correlation evidence is available, the rollback path remains valid, and the documentation has been updated if the contract or operating procedure changed.